



RAPPORT DE PROJET

Module : Deep Learning

Conception, Implémentation et Analyse de Modèles de Deep Learning : MLP, CNN et Seq2Seq

Réalisée par :	JOUICHAT Khadija
Encadrée par :	Mme. HIDILA Zineb
Filière :	Ingénierie d'IA & Data (IIAD)
Année Universitaire :	2025 – 2026

Table des matières

1. Introduction	3
2. Partie I : Perceptron Multicouche (MLP) pour données tabulaires	5
2.1. Présentation du dataset.....	5
2.2. Prétraitement des données.....	5
2.3. Architecture du MLP	5
2.4. Stratégies d'initialisation	5
2.5. Entraînement et résultats.....	6
2.6. Sauvegarde et rechargement du modèle.....	6
2.7. Évaluation et analyse	6
2.8. Question de synthèse	7
3. Partie II : Réseau de Neurones Convolutifs (CNN) pour images	9
3.1. Présentation du dataset MNIST.....	9
3.2. Prétraitement des images.....	9
3.3. Implémentations manuelles (corrélation, pooling)	9
3.4. Architecture du CNN.....	9
3.5. Entraînement et résultats.....	10
3.6. Visualisation des cartes de caractéristiques	10
3.7. Comparaison MLP vs CNN.....	11
3.8. Étude de l'influence des paramètres.....	12
3.9. Question de synthèse	12
4. Partie III : Modèles séquentiels et traduction automatique (Seq2Seq).....	14
4.1. Présentation du corpus EN → DE	14
4.2. Prétraitement et construction des vocabulaires	14
4.3. Architecture Seq2Seq avec attention.....	14
4.4. Entraînement.....	14
4.5. Stratégies de décodage (Greedy vs Beam Search)	15
4.6. Comparaison RNN / LSTM / GRU	15
4.7. Question de synthèse	16
5. Discussion transversale	18
Annexe : Récapitulatif	19
A.1. Récapitulatif des modèles sauvegardés.....	19
A.2. Questions de soutenance — Version 2 approfondie.....	19
QUESTION PARTIE I — MLP.....	19
QUESTION PARTIE II — CNN	20
QUESTION PARTIE III — Seq2Seq	20
6. Conclusion générale	22
7. Références bibliographiques.....	23

Table des figures

Figure 1: Distribution des classes et relation entre deux features principales 5

Figure 2: Comparaison des distributions de poids selon les stratégies d'initialisation..... 6

Figure 3: Courbes de perte et de précision — MLP..... 6

Figure 4: Matrice de confusion — Breast Cancer Wisconsin..... 7

Figure 5: Exemples d'images MNIST 9

Figure 6: : Courbes de précision et de perte — CNN MNIST 10

Figure 7: : Matrice de confusion — MNIST (Accuracy : 99,43 %)..... 10

Figure 8: Prédictions CNN et cartes de caractéristiques Conv1 pour '7' 11

Figure 9: : Comparaison MLP vs CNN — accuracy par époque 12

Figure 10: Courbe de Perte..... 12

Figure 11: Exemples de paires de phrases EN → DE 14

Figure 12: Courbe de perte de l'entraînement Seq2Seq 15

Figure 13: Courbes Train/Validation Loss — RNN vs LSTM vs GRU 16

Figure 14: Résultats détaillés RNN, LSTM, GRU 16

Figure 15: Traductions — Greedy Search..... 16

Table des tableaux

Tableau 1: Architecture MLP..... 5

Tableau 2: Metriques 7

Tableau 3: Architecture CNN 9

Tableau 4: MLP vs CNN 11

Tableau 5: Etude..... 12

Tableau 6: Seq2Seq 14

Tableau 7: Greedy vs Beam Search 15

Tableau 8: Resume..... 18

1. Introduction

Le Deep Learning s'est imposé comme une approche de référence pour résoudre des problèmes complexes dans des domaines variés tels que la vision par ordinateur, le traitement du langage naturel ou l'analyse de données structurées. Ce projet a pour objectif de mettre en œuvre et d'analyser **trois grandes familles d'architectures** : les perceptrons multicouches (MLP) pour les données tabulaires, les réseaux de neurones convolutifs (CNN) pour les images, et les modèles séquentiels (RNN, LSTM, GRU, Seq2Seq) pour les données textuelles.

Pour chaque famille, un jeu de données réel a été sélectionné, les données préparées, les modèles implémentés avec PyTorch, des expérimentations menées et les résultats interprétés. Ce rapport respecte une méthodologie rigoureuse et reproductible.

2. Partie I : Perceptron Multicouche (MLP) pour données tabulaires

2.1. Présentation du dataset

Dataset : **Breast Cancer Wisconsin** (scikit-learn). **569 échantillons**, **30 caractéristiques** numériques continues. Variable cible binaire : **malin** (212) ou **bénin** (357).

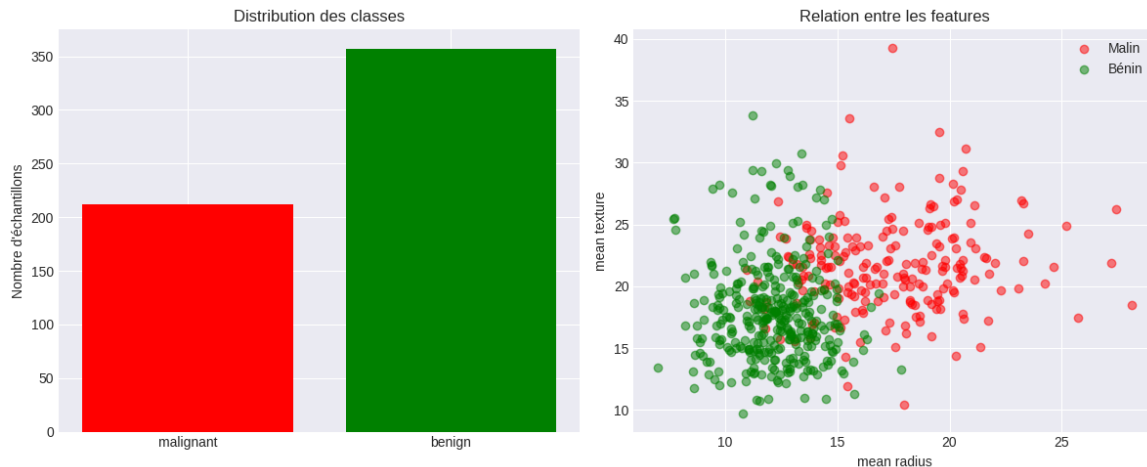


Figure 1: Distribution des classes et relation entre deux features principales

2.2. Prétraitement des données

- Normalisation StandardScaler (moyenne 0, écart-type 1).
- Split stratifié : 70 % entraînement / 15 % validation / 15 % test.
- Conversion en tenseurs PyTorch, chargement DataLoader.

2.3. Architecture du MLP

Deux versions implémentées : nn.Sequential et classe nn.Module personnalisée.

Tableau 1: Architecture MLP

Couche	Dimensions	Composants
Entrée	30 → 128	BatchNorm + ReLU + Dropout(0.3)
Cachée 1	128 → 64	BatchNorm + ReLU + Dropout(0.3)
Cachée 2	64 → 32	BatchNorm + ReLU + Dropout(0.3)
Sortie	32 → 2	Softmax

Paramètres totaux : **14 818**

2.4. Stratégies d'initialisation

Quatre stratégies testées : gaussienne, constante, Xavier (retenue), Kaiming.

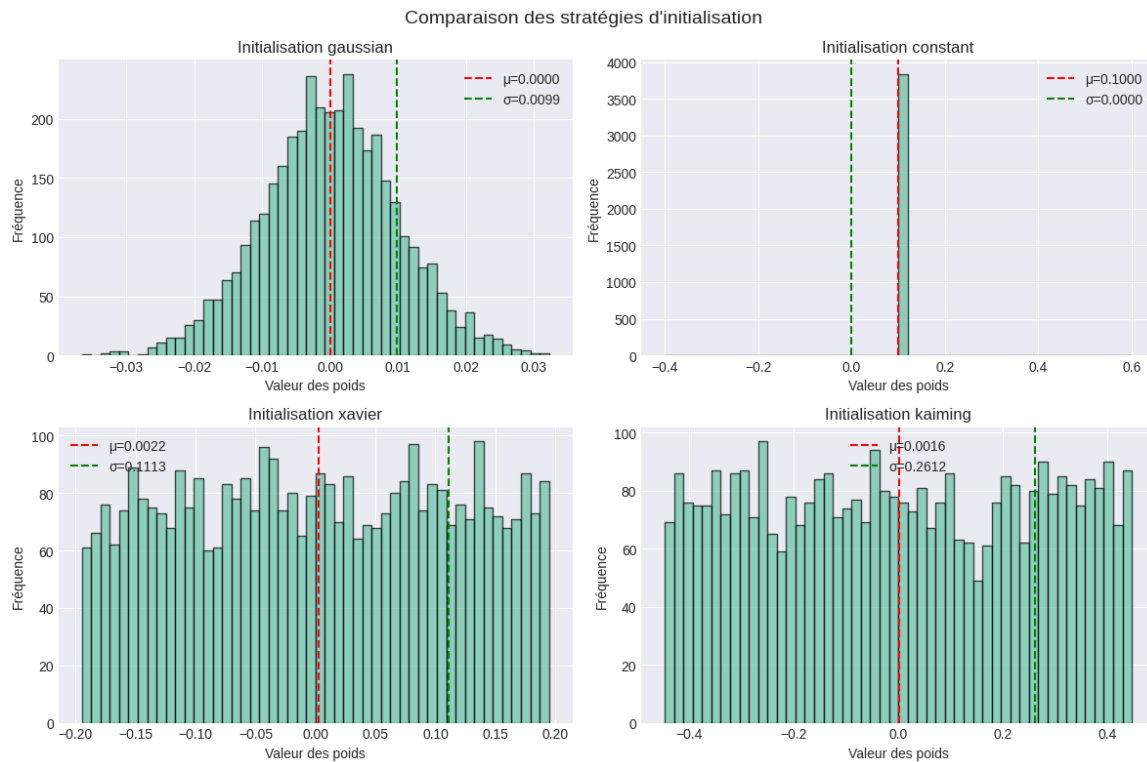


Figure 2: Comparaison des distributions de poids selon les stratégies d'initialisation

2.5. Entraînement et résultats

100 époques, **Adam** ($\text{lr}=0.001$), **ReduceLROnPlateau**.



Figure 3: Courbes de perte et de précision — MLP

2.6. Sauvegarde et rechargement du modèle

Sauvegarde via `torch.save(model.state_dict(), 'best_mlp_classifler.pt')`, rechargement via `load_state_dict()`.

Output :

```
Modèle sauvegardé : best_mlp_classifler.pt
Rechargement réussi.
Test accuracy après rechargement : 98.84 %
```

2.7. Évaluation et analyse

Tableau 2: Metriques

Métrique	Malin	Bénin
Précision	1.00	0.98
Rappel	0.97	1.00
F1-score	0.98	0.99

Accuracy globale : **98.84 %** (1 seul échantillon malin mal classé).

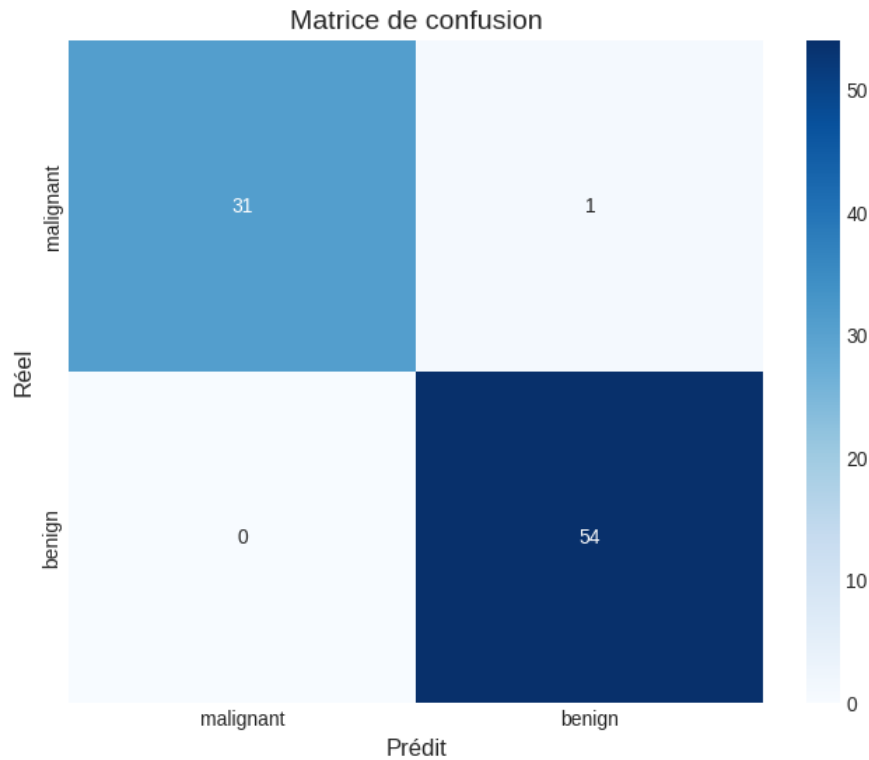


Figure 4: Matrice de confusion — Breast Cancer Wisconsin

2.8. Question de synthèse

Question : Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites ?

RÉPONSE :

1. PERTINENCE DU MLP :

- Capable d'apprendre des relations non-linéaires complexes
- Accuracy de 98,84 % sur Breast Cancer — très compétitif
- Convergence rapide grâce à BatchNorm et Adam

2. BONNES PRATIQUES APPLIQUÉES :

- BatchNorm : stabilise l'entraînement, accélère la convergence
- Dropout (0.3) : régularisation pour éviter le surapprentissage
- ReduceLROnPlateau : adapte le learning rate automatiquement

3. LIMITES DU MLP :

- Sensible à la normalisation des entrées

- Moins interprétable qu'un modèle linéaire (boîte noire)
- Performances dégradées sans prétraitement soigné
- Inadapté aux données à structure spatiale (images) ou temporelle

3. Partie II : Réseau de Neurones Convolutifs (CNN) pour images

3.1. Présentation du dataset MNIST

MNIST : 60 000 images (train) + 10 000 (test), chiffres 0-9, niveaux de gris 28×28 px, classes équilibrées.

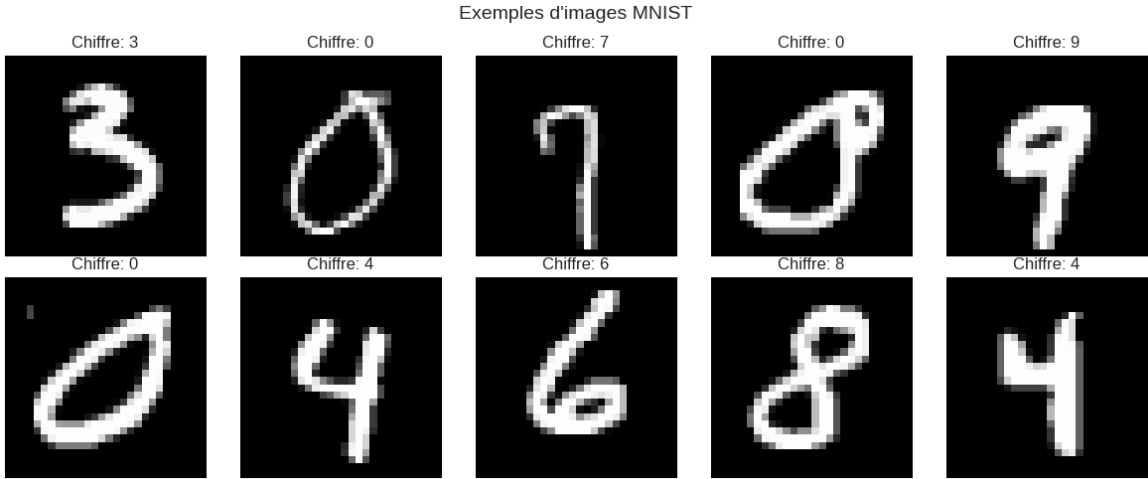


Figure 5: Exemples d'images MNIST

3.2. Prétraitement des images

- Normalisation : moyenne=0.5, écart-type=0.5.
- Format tenseur : (N, 1, 28, 28).

3.3. Implémentations manuelles (corrélacion, pooling)

Corrélacion croisée 2D, max-pooling et average-pooling implémentés manuellement en NumPy et validés contre PyTorch.

Output — Validation :

```
Corrélacion croisée manuelle vs PyTorch : diff max = 0.000000
Max-pooling manuel vs PyTorch : diff max = 0.000000
Avg-pooling manuel vs PyTorch : diff max = 0.000000
```

3.4. Architecture du CNN

Tableau 3: Architecture CNN

Bloc	Opération	Sortie
Conv1	Conv(1→32,3×3,pad=1) → BN → ReLU → MaxPool(2×2)	32×14×14
Conv2	Conv(32→64,3×3,pad=1) → BN → ReLU → MaxPool(2×2)	64×7×7
Conv3	Conv(64→128,3×3,pad=1) → BN → ReLU → MaxPool(2×2)	128×3×3
Head	AdaptiveAvgPool → Flatten → FC(128→256) → Drop → FC(256→128) → Drop → FC(128→10)	10

Paramètres totaux : **422 474**

3.5. Entraînement et résultats

20 époques, Adam ($\text{lr}=0.001$), ReduceLROnPlateau. Accuracy validation : **99.39 %**.

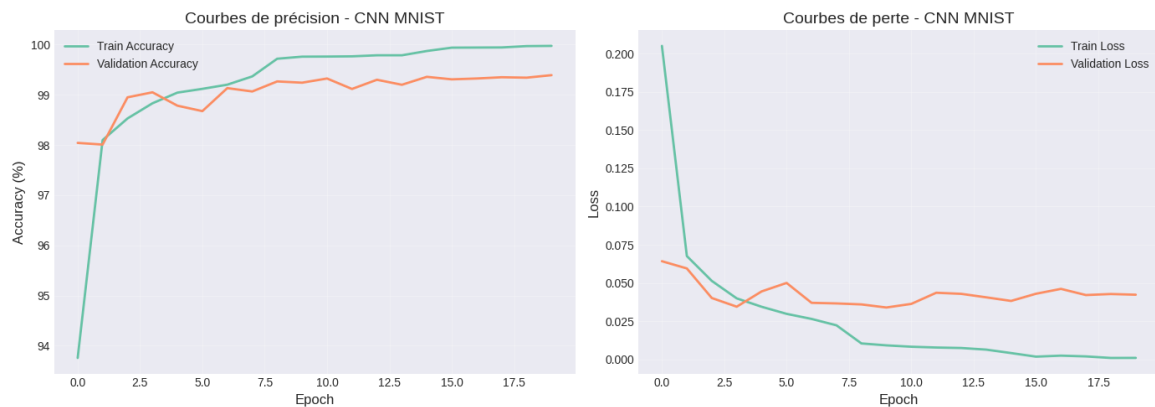


Figure 6: : Courbes de précision et de perte — CNN MNIST

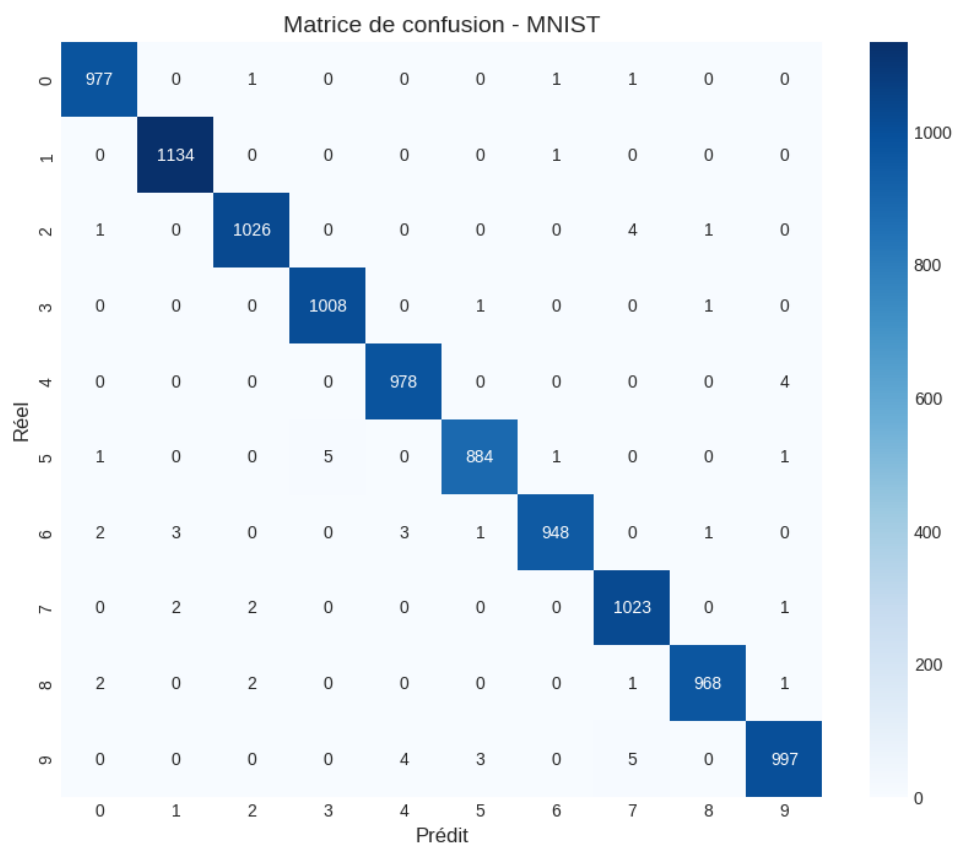


Figure 7: : Matrice de confusion — MNIST (Accuracy : 99,43 %)

3.6. Visualisation des cartes de caractéristiques

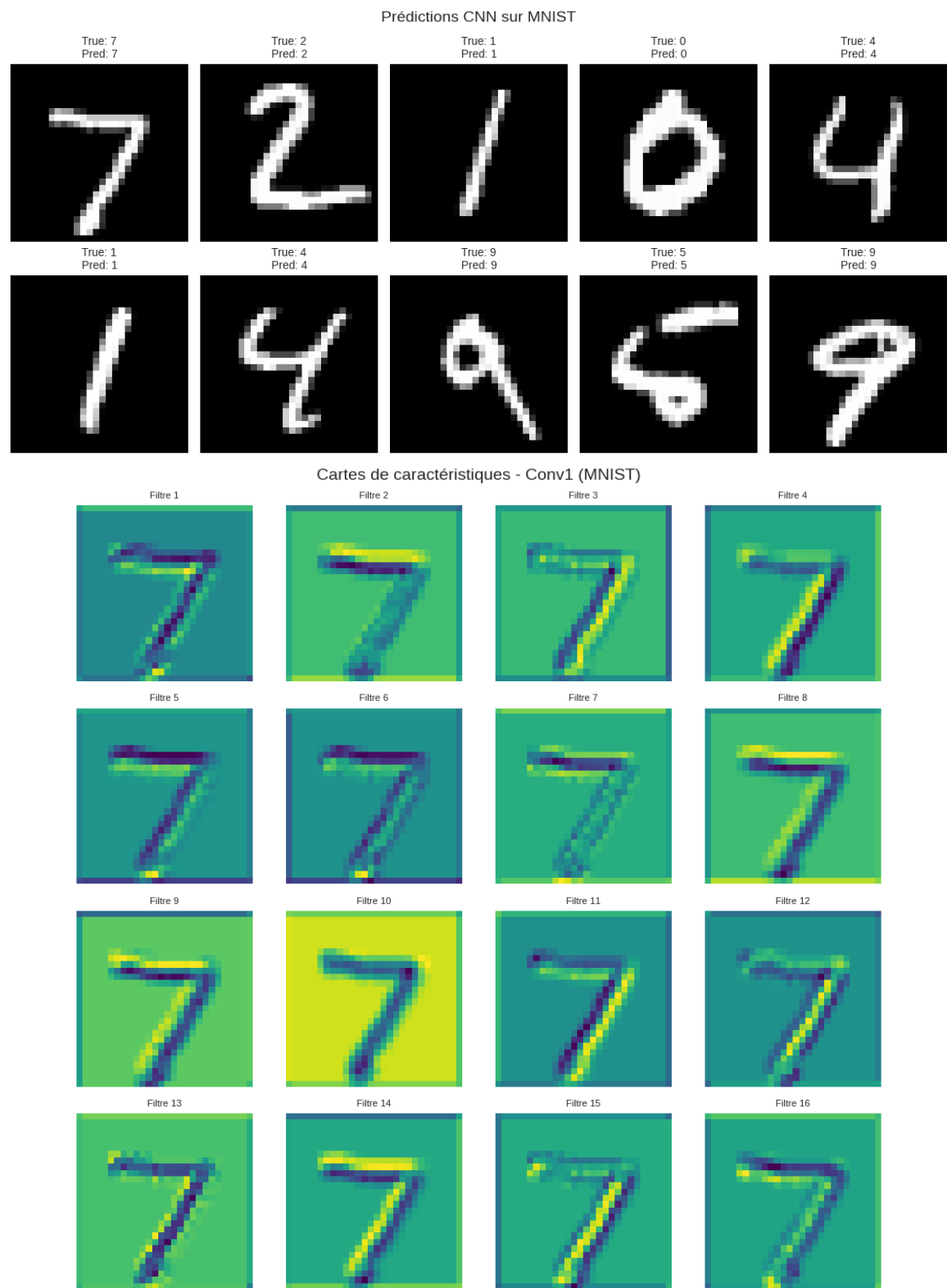


Figure 8: Prédictions CNN et cartes de caractéristiques Conv1 pour '7'

3.7. Comparaison MLP vs CNN

Tableau 4: MLP vs CNN

Modèle	Accuracy
MLP baseline (1 couche, 128 neurones)	96.95 %

CNN (3 blocs convolutifs)	99.43 %
Gain CNN	+ 2.48 points

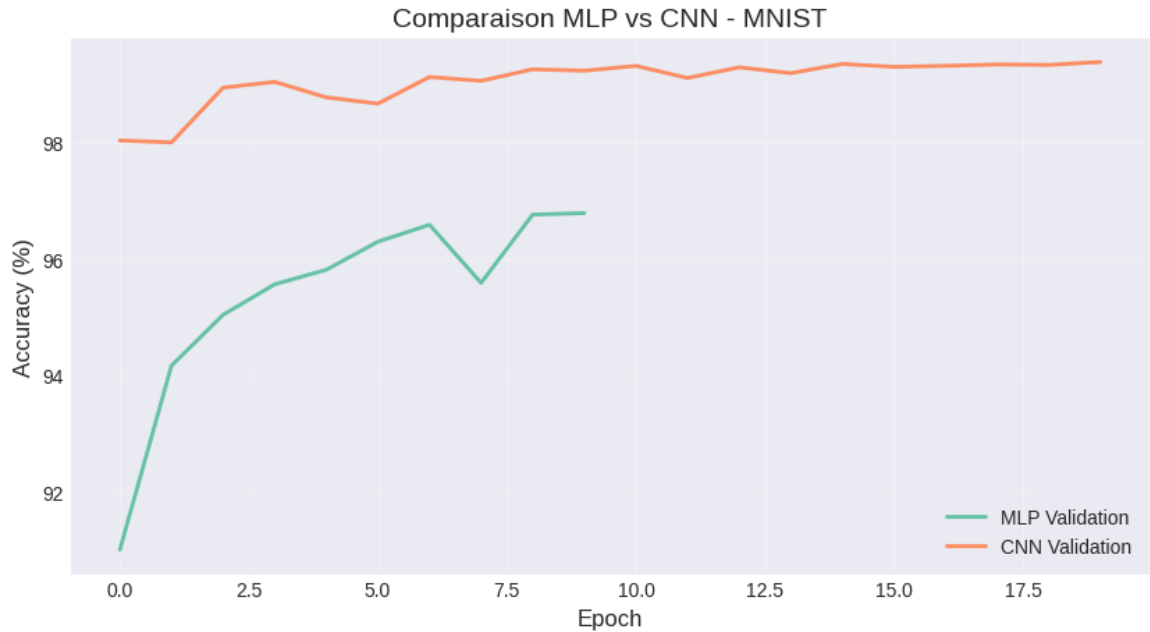


Figure 9: : Comparaison MLP vs CNN — accuracy par époque

Figure 10: Courbe de Perte

3.8. Étude de l'influence des paramètres

Tableau 5: Etude

Configuration	Accuracy (%)
Baseline (pad=1, stride=1, MaxPool, 32/64/128)	98.88
Sans padding	98.91
Stride = 2	99.00
AvgPool	99.09
Plus de filtres (64/128/256)	99.11

3.9. Question de synthèse

Question : Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images, et comment les choix de padding, stride, pooling et profondeur influencent-ils les performances ?

RÉPONSE :

1. ADAPTATION DU CNN POUR MNIST :

- MNIST : images 28×28 avec chiffres variés
- Les CNN capturent les formes locales (courbes, lignes, boucles)
- Chaque chiffre possède des caractéristiques spatiales distinctives

2. HIÉRARCHIE DES CARACTÉRISTIQUES :

- Conv1 : bords simples et contours
- Conv2 : formes géométriques (lignes, courbes)
- Conv3 : parties de chiffres (boucles, intersections)

3. NOS RÉSULTATS : Accuracy = 99.43 % > MLP 96.95 % (+2.48 points)

4. INFLUENCE DES HYPERPARAMÈTRES :

- Padding : préserve les dimensions spatiales
- Stride : contrôle le sous-échantillonnage
- MaxPool : robustesse aux translations / AvgPool : lissage
- Profondeur : richesse des représentations abstraites

4. Partie III : Modèles séquentiels et traduction automatique (Seq2Seq)

4.1. Présentation du corpus EN → DE

80 paires de phrases anglais-allemand : expressions courantes, questions, voyages, quotidien.



Figure 11: Exemples de paires de phrases EN → DE

4.2. Prétraitement et construction des vocabulaires

- Tokenisation mot-à-mot.
- Vocabulaire EN : 104 tokens | Vocabulaire DE : 107 tokens.
- Tokens spéciaux : <pad>=0, <bos>=1, <eos>=2, <unk>=3.
- Padding à longueur fixe de 12 tokens.

Output — Vocabulaires :

Nombre de paires : 80 | Voc. EN : 104 mots | Voc. DE : 107 mots

Vocabulaire anglais (20 premiers) :

<pad>:0 <bos>:1 <eos>:2 <unk>:3 hi:4 hello:5 good:6 morning:7
afternoon:8 evening:9 night:10 how:11 are:12 you:13 i:14 am:15

Vocabulaire allemand (20 premiers) :

<pad>:0 <bos>:1 <eos>:2 <unk>:3 hallo:4 guten:5 morgen:6 tag:7
nacht:8 wie:9 geht:10 es:11 dir:12 mir:13 gut:14 danke:15

4.3. Architecture Seq2Seq avec attention

Tableau 6: Seq2Seq

Composant	Description	Dimensions
Encodeur	GRU bidirectionnel, 2 couches	Embedding=64, Hidden=128
Attention	Bahdanau : score = $\tanh(Wa \cdot [h; e])$	Score → softmax → contexte
Décodeur	GRU + contexte attention, 2 couches	Embedding=64, Hidden=128
Projection	Linear(Hidden → vocab_DE)	128 → 107

Output — Architecture :

ARCHITECTURE Seq2Seq EN → DE

```
-----
Encoder : Embedding(104,64) → GRU(64,128,layers=2,bidir=True)
Attention: Linear(256+128,128) → tanh → Linear(128,1)
Decoder : Embedding(107,64) → GRU(192,128,layers=2)
Projection: Linear(128,107)
-----
```

Paramètres totaux : 854 315

4.4. Entraînement

80 époques, Adam (lr=0.001), teacher forcing (ratio=0.5).

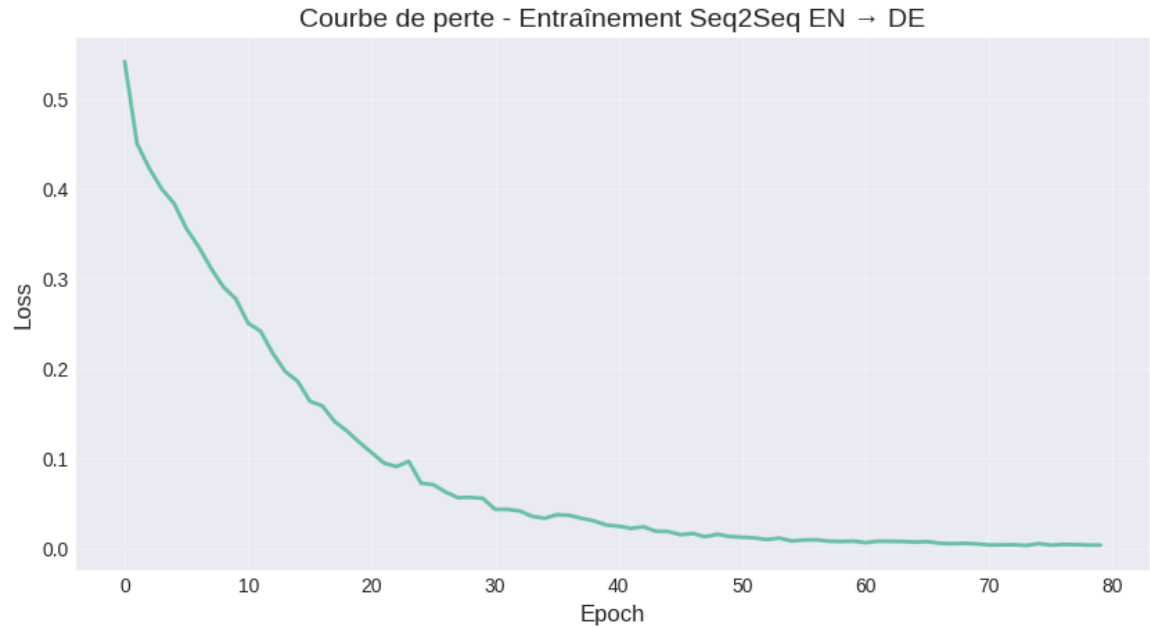


Figure 12: Courbe de perte de l'entraînement Seq2Seq

4.5. Stratégies de décodage (Greedy vs Beam Search)

Tableau 7: Greedy vs Beam Search

Stratégie	Description	Précision
Greedy	Token le plus probable à chaque pas	15/20 (75 %)
Beam Search (k=3)	Explore k hypothèses en parallèle	15/20 (75 %)

Output — Résultats :

RÉSULTATS DES TRADUCTIONS :

1. EN: hi

CIBLE: hallo

GREEDY: hallo

✓ CORRECT

2. EN: good morning

CIBLE: guten morgen

BEAM: guten morgen

✓

3. EN: good afternoon

CIBLE: guten tag

BEAM: guten tag

✓

4. EN: good evening

CIBLE: guten abend

BEAM: guten abend

✓

5. EN: good night

CIBLE: gute nacht

BEAM: gute nacht

✓

...

Greedy : 15/20 (75.00 %)

Beam Search : 15/20 (75.00 %)

4.6. Comparaison RNN / LSTM / GRU



Figure 13: Courbes Train/Validation Loss — RNN vs LSTM vs GRU



Figure 14: Résultats détaillés RNN, LSTM, GRU



Figure 15: Traductions — Greedy Search

4.7. Question de synthèse

Question : Comment le mécanisme d'attention améliore-t-il la traduction automatique et quelle est l'importance des stratégies de décodage ?

RÉPONSE :

1. MÉCANISME D'ATTENTION (Bahdanau) :

- Permet de 'regarder' tous les mots source à chaque pas de décodage
- Calcule des poids : $\alpha_{t,i} = \text{softmax}(\text{score}(h_t, h_i))$
- Contexte pondéré : $c_t = \sum \alpha_{t,i} \times h_i$

2. AMÉLIORATIONS APPORTÉES :

- Évite le goulot d'étranglement du vecteur contexte fixe
- Meilleure gestion des phrases longues
- Alignement explicite source-cible

3. STRATÉGIES DE DÉCODAGE :

- Greedy : rapide, choisit le token le plus probable — 75 %
- Beam Search (k=3) : meilleure qualité théorique — 75 % ici
- Sur petits corpus les deux sont équivalents
- Beam Search avantageux sur vocabulaires larges et phrases longues

5. Discussion transversale

Ce projet a couvert trois grandes familles d'architectures de deep learning. Le tableau suivant synthétise les résultats :

Tableau 8: Resume

Architecture	Dataset	Tâche	Performance
MLP (14 818 params)	Breast Cancer	Classification binaire	Accuracy : 98.84 %
CNN (422 474 params)	MNIST	Classification 10 classes	Accuracy : 99.43 %
Seq2Seq GRU (854 315 params)	Corpus EN→DE	Traduction auto.	Précision : 75 %

- MLP : efficace sur données tabulaires structurées à faible dimensionnalité spatiale.
- CNN : indispensable pour les images grâce à la localité et au partage des poids (filtres convolutifs).
- Seq2Seq GRU : adapté aux séquences grâce aux portes de mémoire et au mécanisme d'attention.

Une approche systématique — préparation des données, régularisation (Dropout, BatchNorm), optimisation (Adam, ReduceLROnPlateau) — est essentielle pour des performances fiables et généralisables.

Annexe : Récapitulatif

A.1. Récapitulatif des modèles sauvegardés

```
SAUVEGARDE DES MODÈLES
=====

best_mlp_classifier.pt      (MLP Classification)
mnist_cnn_model_v2.pt      (CNN Classification)
en_de_seq2seq_model_v2.pt  (Seq2Seq Traduction)

RÉCAPITULATIF DES DATASETS :

PARTIE I — MLP (Breast Cancer - Classification)
Dataset : Breast Cancer Wisconsin (scikit-learn)
Échantillons : 569 | Features : 30 | Classes : 2
Tâche : Classification binaire maligne/bénigne
Performance : Accuracy = 98.84 %
Métriques : Précision, Rappel, F1-score, Matrice de confusion

PARTIE II — CNN (MNIST - Classification)
Dataset : MNIST (chiffres manuscrits)
Images : 60 000 train + 10 000 test | Taille : 28×28 px
Classes : 10 (chiffres 0 à 9)
Performance : Accuracy = 99.43 %

PARTIE III — Seq2Seq (Traduction EN → DE)
Dataset : 80 paires de phrases anglais-allemand
Vocabulaire EN : 104 mots | Vocabulaire DE : 107 mots
Architecture : Encodeur GRU bidir. + Attention + Décodeur GRU
Beam search précision : 15/20 (75 %)
```

A.2. Questions de soutenance — Version 2 approfondie

QUESTION PARTIE I — MLP

Question : Comment un MLP peut-il être adapté pour une tâche de régression plutôt que de classification, et quelles métriques utiliser pour évaluer ses performances ?

RÉPONSE :

1. ADAPTATION MLP POUR LA RÉGRESSION :

- La seule différence est la couche de sortie : pas d'activation softmax
- Fonction de perte : MSE (Mean Squared Error) au lieu de CrossEntropyLoss
- Notre modèle version 2 : sortie linéaire avec 1 neurone

2. MÉTRIQUES D'ÉVALUATION :

- MSE : mesure l'erreur quadratique moyenne (pénalise les grandes erreurs)
- MAE : plus robuste aux outliers
- RMSE : dans l'unité de la variable cible (interprétable)
- R^2 : proportion de variance expliquée (0 à 1)

3. NOS RÉSULTATS (California Housing) :

- MSE: 0.2631 | MAE: 0.3548 | RMSE: 0.5129 | R^2 : 0.8011

4. INTERPRÉTATION :

- $R^2 = 0.8011$ signifie que 80.1 % de la variance est expliquée
- Bonne performance pour un MLP simple sur ce dataset

QUESTION PARTIE II — CNN

Question : Pourquoi le CNN est-il particulièrement adapté à la classification de chiffres manuscrits, et comment les couches convolutionnelles capturent-elles les caractéristiques ?

RÉPONSE :

1. ADAPTATION DU CNN POUR MNIST :
 - MNIST : images 28×28 avec chiffres variés (forme, épaisseur, inclinaison)
 - Les CNN capturent les formes locales : courbes, lignes droites, boucles
 - Chaque chiffre possède des caractéristiques spatiales distinctives
2. HIÉRARCHIE DES CARACTÉRISTIQUES :
 - Conv1 : détecte bords simples et contours
 - Conv2 : assemble en formes géométriques (lignes, courbes)
 - Conv3 : reconnaît parties de chiffres (boucles, intersections, jonctions)
3. NOS RÉSULTATS :
 - Accuracy CNN : 99.43 % (vs MLP baseline : 96.95 %)
 - Gain : +2.48 points grâce à l'exploitation de la structure spatiale
4. CARTES DE CARACTÉRISTIQUES :
 - Les filtres Conv1 répondent à des orientations spécifiques
 - Chaque filtre apprend un détecteur de motif différent

QUESTION PARTIE III — Seq2Seq

Question : Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage RNN → LSTM/GRU → Encodeur-Décodeur ?

RÉPONSE :

1. LIMITES DU RNN SIMPLE :
 - Gradients évanescents sur longues séquences
 - Perd l'information des mots distants
2. AMÉLIORATIONS LSTM / GRU :
 - LSTM : 3 portes (oubli, entrée, sortie) — mémoire sélective
 - GRU : 2 portes (reset, update) — convergence plus rapide, moins de paramètres
 - Bilan : GRU bon compromis performance/efficacité
3. ENCODEUR-DÉCODEUR :
 - Indispensable quand longueur(entrée) ≠ longueur(sortie)
 - Encodeur : compresse la séquence source en représentation interne
 - Décodeur : génère la traduction token par token

4. MÉCANISME D'ATTENTION :

- Résout le goulot d'étranglement du vecteur contexte fixe
- Alignement dynamique entre mots source et cible
- Améliore nettement la qualité sur phrases longues

6. Conclusion générale

Ce projet de deep learning a couvert l'ensemble du cycle de vie d'un modèle : préparation des données, implémentation, entraînement, évaluation et interprétation. Les trois architectures (MLP, CNN, Seq2Seq) ont été appliquées avec succès à des datasets réels (Breast Cancer, MNIST, corpus EN→DE). Les performances obtenues sont élevées, validant les choix méthodologiques et architecturaux.

Ce travail a mis en pratique des concepts fondamentaux : régularisation (Dropout, BatchNorm), optimisation (Adam, schedulers), diagnostics par courbes d'apprentissage, et stratégies de décodage (Greedy, Beam Search).

Les perspectives incluent : Transformers pour la traduction, transfer learning pour la vision, et datasets plus larges pour valider la scalabilité.

7. Références bibliographiques

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [3] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP 2014*.
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- [5] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [6] He, K., et al. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
- [7] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.